



Universidad del Valle de Guatemala

Facultad de Ingeniería

CC3092 — Deep Learning y Sistemas Inteligentes

Laboratorio #2

Redes Neuronales Convolucionales (CNN)

Clasificación de dígitos manuscritos — MNIST

Nicolás Concuá

Guatemala, 9 de agosto de 2026

1. Investigación: capas de PyTorch para la CNN

nn.Conv2d — aplica una convolución 2D deslizando filtros aprendibles sobre la imagen para extraer características locales (bordes, texturas). Parámetros clave: `in_channels`, `out_channels` (nº de filtros), `kernel_size`, `stride` y `padding`.

nn.MaxPool2d — submuestreo que conserva el valor máximo de cada ventana; reduce la resolución espacial, da invariancia a pequeñas traslaciones y amplía el campo receptivo (`kernel_size`, `stride`).

nn.AvgPool2d — igual que `MaxPool2d` pero conserva el promedio de la ventana; suaviza la representación (`kernel_size`, `stride`).

nn.BatchNorm2d — normaliza cada canal usando la media y varianza del batch, con escala (γ) y desplazamiento (β) aprendibles; estabiliza y acelera el entrenamiento (`num_features` = canales del `Conv2d` previo).

nn.Flatten — aplanar el tensor (N,C,H,W) a un vector (N, C·H·W), conectando las capas convolucionales con las densas (`start_dim`, `end_dim`).

nn.CrossEntropyLoss — pérdida para clasificación multiclase; combina `LogSoftmax` + `NLLLoss`, por lo que recibe los logits crudos y las etiquetas como enteros (`weight`, `label_smoothing`).

Conceptos. Un *tensor* es un arreglo n-dimensional con soporte para GPU/MPS y diferenciación automática; las imágenes se representan como tensores (N,C,H,W). El *campo receptivo* es la región de la entrada que influye en una neurona: pequeño en las primeras capas y creciente con la profundidad y el pooling. Una CNN usa *menos parámetros que un MLP* gracias a la conectividad local (cada neurona ve solo un vecindario) y la compartición de pesos (un mismo filtro se reutiliza en toda la imagen), capturando estructura espacial con una fracción de los pesos que requeriría una capa densa.

2. Resultados de las iteraciones (validación)

Se entrenaron 6 configuraciones por arquitectura (12 en total), variando 1–2 hiperparámetros a la vez, con Adam, batch=128 y 10 epochs. Métricas macro sobre validación (6,000 imágenes).

Iteración	Arq.	Configuración	Drop	WD	Params	Val Acc	Val F1
MLP-1 base	MLP	hidden=[128]	0.0	0	101,770	0.9747	0.9744
MLP-2 wider	MLP	hidden=[256]	0.0	0	203,530	0.9773	0.9771
MLP-3 deeper	MLP	hidden=[256,128]	0.0	0	235,146	0.9763	0.9761
MLP-4 dropout	MLP	hidden=[256,128]	0.3	0	235,146	0.9788	0.9785
MLP-5 drop+wd	MLP	hidden=[256,128]	0.3	1e-4	235,146	0.9785	0.9781
MLP-6 bigger	MLP	hidden=[512,256]	0.3	1e-4	535,818	0.9797	0.9794
CNN-1 base	CNN	ch=[16,32], sin BN	0.0	0	20,490	0.9865	0.9864

Iteración	Arq.	Configuración	Drop	WD	Params	Val Acc	Val F1
CNN-2 wider	CNN	ch=[32,64], sin BN	0.0	0	50,186	0.9875	0.9874
CNN-3 batchnorm	CNN	ch=[32,64], BN	0.0	0	50,378	0.9877	0.9875
CNN-4 bn+drop	CNN	ch=[32,64], BN	0.25	0	50,378	0.9903	0.9902
CNN-5 bn+drop+wd	CNN	ch=[32,64], BN	0.25	1e-4	50,378	0.9887	0.9885
CNN-6 deeper	CNN	ch=[32,64,128], BN	0.25	1e-4	104,650	0.9918	0.9917

3. Comparación de arquitecturas (test)

La mejor configuración de cada arquitectura (por F1 de validación) se evaluó una única vez sobre el conjunto de test (10,000 imágenes).

Modelo	Params	Test Acc	Test Prec	Test Rec	Test F1	Tiempo (s)
MLP (MLP-6 bigger)	535,818	0.9822	0.9821	0.9820	0.9821	26.4
CNN (CNN-6 deeper)	104,650	0.9926	0.9926	0.9924	0.9925	47.0

La CNN supera al MLP en todas las métricas de test usando **aproximadamente 5× menos parámetros** (104,650 vs 535,818): reduce el error de clasificación de ~1.8% a ~0.7%. Es decir, mejor desempeño y mayor eficiencia paramétrica a la vez.

4. Análisis de resultados

Hiperparámetro de mayor impacto. En ambas arquitecturas el mayor impacto positivo fue el dropout (MLP: F1 0.9761→0.9785; CNN: 0.9875→0.9902). En la CNN, profundizar la red (tercer bloque conv.) dio la mejor iteración (0.9917). El mayor impacto negativo fue aumentar la profundidad del MLP sin regularizar (0.9771→0.9761) y aplicar weight decay a la CNN (0.9902→0.9885).

Overfitting / underfitting. Se observó overfitting, sobre todo en el MLP: la brecha entre la pérdida de entrenamiento y la de validación llegaba a 0.06–0.08 sin regularización. Se identificó por esa brecha y por las curvas de validación estancadas mientras las de train seguían bajando. Se mitigó con dropout (brecha reducida a ~0.01–0.03) y con la selección del mejor epoch por F1 (early stopping). No hubo underfitting relevante.

Regularización. El dropout fue el método más efectivo en ambas arquitecturas; el weight decay fue neutro (MLP) o contraproducente (CNN), y BatchNorm actuó más como estabilizador del entrenamiento que como regularizador de precisión.

Errores (matrices de confusión). El MLP comete 178 errores en test y la CNN solo 74. Ambos fallan en dígitos visualmente similares (9↔4, 9↔7 y confusiones alrededor del 6); el dígito 9 es el más problemático en ambos, pero la CNN reduce los errores en todas las clases.

5. Conclusiones

- La CNN es claramente superior al MLP en MNIST: 0.9926 vs 0.9822 de accuracy en test, con ~5 por menos parámetros, por su capacidad de explotar la estructura espacial mediante conectividad local y compartición de pesos.
- El dropout fue la técnica de regularización más efectiva para ambas arquitecturas y la clave para controlar el overfitting, especialmente marcado en el MLP.
- Para un despliegue en producción se elegiría la CNN: ofrece mayor exactitud y menor tamaño de modelo; su único costo es un entrenamiento/inferencia algo más lentos, irrelevante a la escala de MNIST.

Repositorio: <https://github.com/nicoCT04/Lab2-CNN-DL>